Unit IV: Message Authentication and Hash Functions:

Authentication Requirement, Authentication Function, Message Authentication Code, Hash Function, Security of Hash Function and MACs.

Hash and MAC Algorithm: Secure Hash Algorithm, Whirlpool, HMAC, CMAC.

Digital Signature: Digital Signature, Authentication Protocol, Digital Signature Standard.

Message Authentication Requirements

In the context of communications across a network, the following attacks can be identified:

1. Disclosure-Release of message contents to any person or process not possessing the appropriate cryptographic key.

2. Traffic Analysis-Discovery of the pattern of traffic between parties. In a connection-oriented application, the opponent can determine the frequency and duration of connections, and the number and length of messages between parties. In a connectionless environment, the opponent can determine the number and length of messages between parties.

3. Masquerade-Insertion of messages by an opponent that are from an unauthorized entity. This includes the creation of messages by an opponent that appear to come from an authorized entity. Also included are fraudulent acknowledgments of message receipt or nonreceipt by someone other than the legitimate recipient.

4. Content Modification-Changes to the contents of a message, including insertion, deletion, transposition, and modification.

5. Sequence Modification-Any modification to a sequence of messages between parties, including insertion, deletion, and reordering.

6. Timing Modification-Delay or replay of messages. In a connection-oriented application, an entire session or sequence of messages could be delayed or replayed. A replay of some previous valid session, or individual messages in the sequence, could be delayed or replayed. In a connectionless application, an individual message (e.g., datagram) could be delayed or replayed.

7. Source Repudiation-Denial of transmission of message by source.

8. Destination Repudiation-Denial of receipt of message by destination.

Message Authentication-Message authentication is a procedure to verify that received messages come from the alleged source and have not been altered.

Message authentication may also verify: **Sequencing** and **Timeliness**

A **digital signature** is an authentication technique that also includes measures to counter **repudiation**.

Message Authentication Functions:

Any message authentication or digital signature mechanism has two levels of functionality. At the lower level, there must be some sort of function that produces an authenticator: a value to be used

to authenticate a message. This lower-level function is then used as a primitive in a higher-level authentication protocol that enables a receiver to verify the authenticity of a message.

The types of functions that may be used to produce an authenticator may be grouped into three classes:

Hash function

Message encryption

Message authentication code (MAC)

Hash function: A function that maps a message of any length into a fixed-length hash value, which serves as the authenticator.

Message encryption: The ciphertext of the entire message serves as its authenticator.

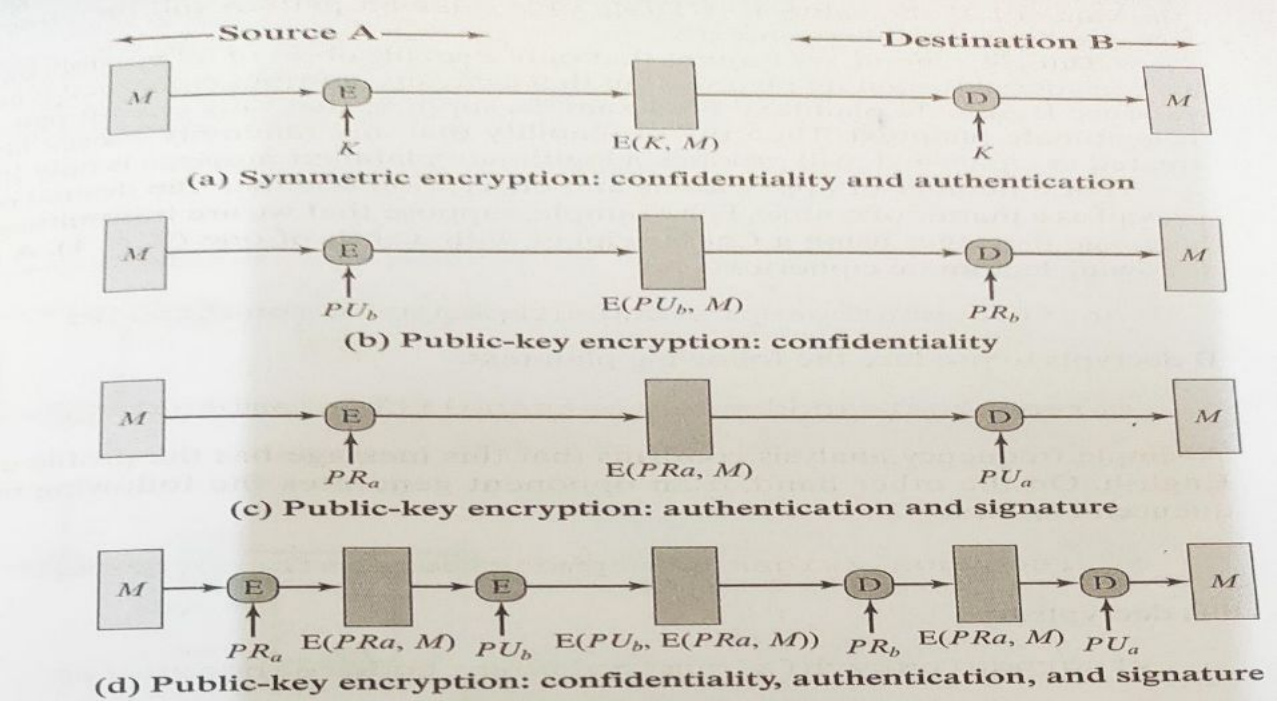
Message authentication code (MAC): A function of the message and a secret key that produces a fixed-length value that serves as the authenticator.

Message Encryption-Message encryption by itself can provide a measure of authentication. The analysis differs for symmetric and public-key encryption schemes.

Symmetric Encryption- Consider the straightforward use of symmetric encryption. A message M transmitted from source A to destination B is encrypted using a secret key K shared by A and B. If no other party knows the key, then confidentiality is provided: no other party can recover the plaintext of the message.

In addition, B is assured that the message was generated by A. The message must have come from A because A is the only other party that possesses K and therefore the only other party with the information necessary to construct ciphertext that can be decrypted with K.

Furthermore, if M is recovered, B knows that none of the bits of M have been altered, because an opponent that does not know the key would not know how to alter bits in the ciphertext to produce the desired change in the plaintext. Thus, symmetric encryption provides authentication as well as confidentiality.



Authentication Using Symmetric Encryption-The above statement needs to be qualified. Consider what is happening at B. Given a decryption function D and a secret key K, the destination will accept any input X and produce: Y=D(K,X). If X is the ciphertext of a legitimate message M produced by the corresponding encryption function, then Y is some legitimate plaintext message M. Otherwise, Y will likely be a meaningless sequence of bits. There may need to be some automated means of determining at B whether Y is legitimate plaintext and therefore must have come from A.

Requirement for Authentication-Suppose the message M can be an arbitrary bit pattern. In that case, there is no way to determine automatically, at the destination, whether an incoming ciphertext is the ciphertext of a legitimate message. This conclusion is incontrovertible. If M can be any bit pattern, then regardless of the value of X, Y=D(K,X) is some bit pattern and therefore must be accepted as authentic plaintext. Thus, in general, we require that only a small subset of all possible bit patterns be considered legitimate plaintext. In that case, any spurious ciphertext is unlikely to produce legitimate plaintext. For example, suppose that only one bit pattern in 10^6 is legitimate plaintext. Then the probability that any randomly chosen bit pattern, treated as ciphertext, will produce a legitimate plaintext message is only: 10^{-6}

Example: Caesar Cipher, For a number of applications and encryption schemes, the desired conditions prevail as a matter of course. For example, suppose that we are transmitting English language messages using a Caesar cipher with a shift of one (K=1).

A sends the following legitimate ciphertext: nbsftfbupbutboeepftfbupbutboemjuumrfmnbctfbu jwz

B decrypts to produce the following plaintext: mareseatoatsanddoeseatoatsandlittlelambseativy

A simple frequency analysis confirms that this message has the profile of ordinary English. On the other hand, if an opponent generates the following random sequence of letters:

zuvrsoevegqx1zwigamdvrnmhpmccxiureosfbcetqxsxq

this decrypts to: ytuqrndufpwkyvhfzlcmlgolbwh ttqdnreadbaspwrwp

which does not fit the profile of ordinary English.

STRUCTURE IN PLAINTEXT-It may be difficult to determine automatically if incoming ciphertext decrypts to unintelligible plaintext. If the plaintext is, say, a binary object file or digitized X-rays, determination of properly formed and therefore authentic plaintext may be difficult. Thus, an opponent could achieve a certain level of disruption simply by issuing messages with random content purporting to come from a legitimate user. One solution to this problem is to force the plaintext to have some structure that is easily recognized but that cannot be replicated without recourse to the encryption function.

ERROR-DETECTING CODE-We could, for example, append an error-detecting code, also known as a frame check sequence (FCS) or checksum, to each message before encryption.

A prepares a plaintext message M and then provides this as input to a function F that produces an FCS. The FCS is appended to M and the entire block is then encrypted. At the destination, B decrypts the incoming block and treats the result as a message with an appended FCS. B applies the same function F to attempt to reproduce the FCS. If the calculated FCS is equal to the incoming FCS, then the message is considered authentic. It is unlikely that any random sequence of bits would exhibit the desired relationship.

INTERNAL ERROR CONTROL-The order in which the FCS and encryption functions are performed is critical. The sequence illustrated in the figure is referred to as internal error control.With internal error control, authentication is provided because an opponent would have difficulty generating ciphertext that, when decrypted, would have valid error-control bits.

EXTERNAL ERROR CONTROL-The authors contrast internal error control with external error control . If instead the FCS is the outer code, an opponent can construct messages with valid error-control codes. Although the opponent cannot know what the decrypted plaintext will be, he or she can still hope to create confusion and disrupt operations.

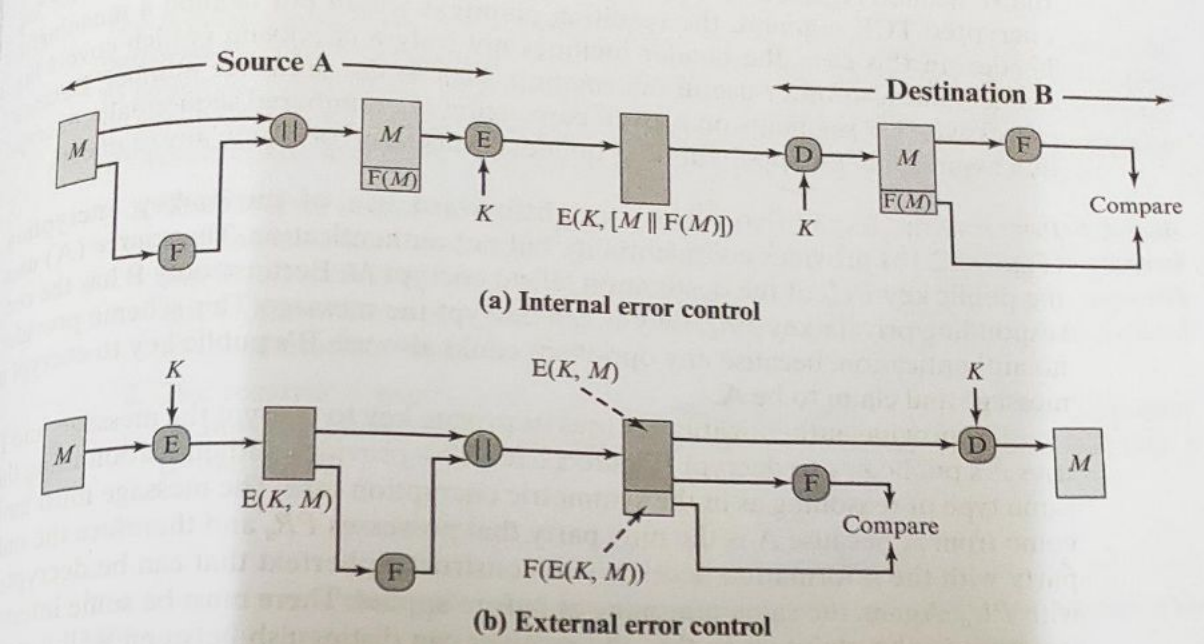


Figure 12.2 Internal and External Error Control

Internal error control:- The FCS is generated first, appended to the message, and the complete block is encrypted.

External error control: The FCS is generated from the encrypted message and is placed outside the encryption.

STRUCTURE PROVIDED BY COMMUNICATION PROTOCOLS-An error-control code is just one example; in fact, any sort of structure added to the transmitted message serves to strengthen the authentication capability. Such structure is provided by the use of a communications architecture consisting of layered protocols. As an example, consider the structure of messages transmitted using the TCP/IP protocol architecture.

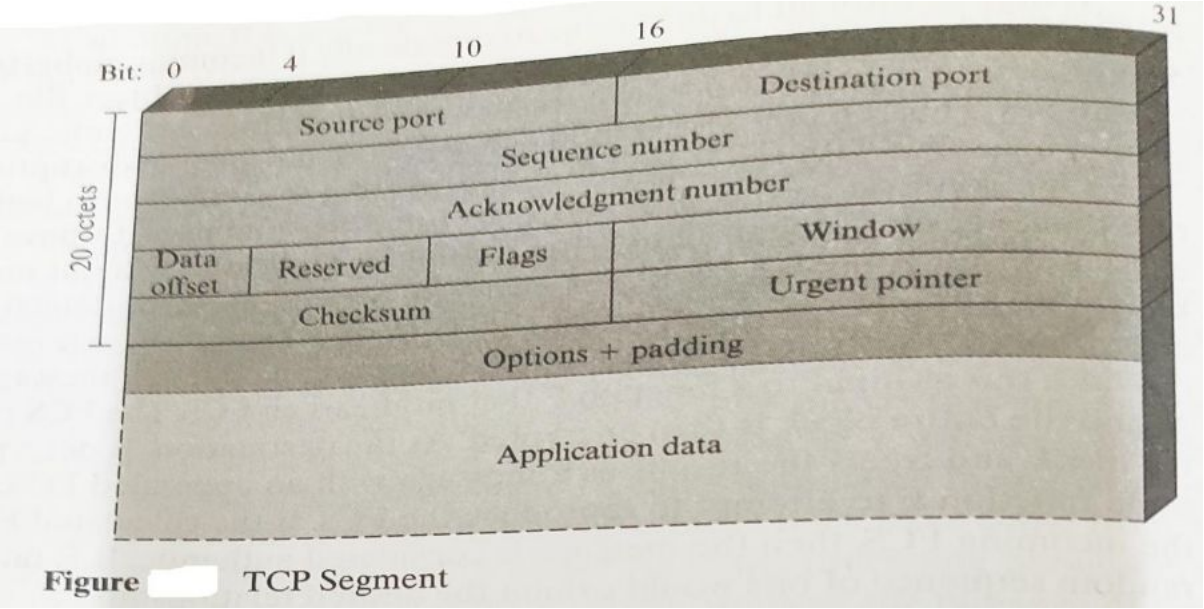


Figure TCP Segment

TCP/IP STRUCTURE AND AUTHENTICATION-Now suppose that each pair of hosts shared a unique secret key, so that all exchanges between a pair of hosts used the same key, regardless of application. Then we could simply encrypt all of the datagram except the IP header. Again, if an opponent substituted some arbitrary bit pattern for the encrypted TCP segment, the resulting plaintext would not include a meaningful header. In this case, the header includes not only a checksum but also other useful information, such as the sequence number. Because successive TCP segments on a given connection are numbered sequentially, encryption assures that an opponent does not delay, misorder, or delete any segments.

PUBLIC-KEY ENCRYPTION-The straightforward use of public-key encryption provides confidentiality but not authentication. The source A uses the public key: PU_b of the destination B to encrypt M. Because only B has the corresponding private key PR_b. only B can decrypt the message. This scheme provides no authentication, because any opponent could also use B's public key to encrypt a message and claim to be A.

PUBLIC-KEY ENCRYPTION FOR AUTHENTICATION- To provide authentication, A uses its private key to encrypt the message, and B uses A's public key to decrypt. This provides authentication using the same type of reasoning as in the symmetric encryption case. The message must have come from A because A is the only party that possesses: PRa and therefore the only party with the information necessary to construct ciphertext that can be decrypted with: PUa Again, the same reasoning as before applies: There must be some internal structure to the plaintext so that the receiver can distinguish between well-formed plaintext and random bits.

DIGITAL SIGNATURE-Assuming there is such structure, then the scheme of Figure (c) does provide authentication. It also provides what is known as a digital signature. Only A could have constructed the ciphertext because only A possesses PRa. Not even B, the recipient, could have constructed the ciphertext, because B has the means to prove that the message must have come from A. In effect, A has “signed” the message by using its private key to encrypt. This scheme does not provide confidentiality. Anyone in possession of A's public key can decrypt the ciphertext.

CONFIDENTIALITY AND AUTHENTICATION-To provide both confidentiality and authentication, A can encrypt M first using its private key, which provides the digital signature, and then using B's public key, which provides confidentiality. The disadvantage of this approach is that the public-key algorithm, which is complex, must be exercised four times rather than two in each communication.

MESSAGE AUTHENTICATION CODE

An alternative authentication technique involves the use of a secret key to generate a small fixed-size block of data, known as a cryptographic checksum or MAC, that is appended to the message. This technique assumes that two communicating parties, say A and B, share a common secret key K.

When A has a message to send to B, A calculates the MAC as a function of the message and the key:
MAC =C (K,M)

where:

M = input message

C = MAC function

K = shared secret key

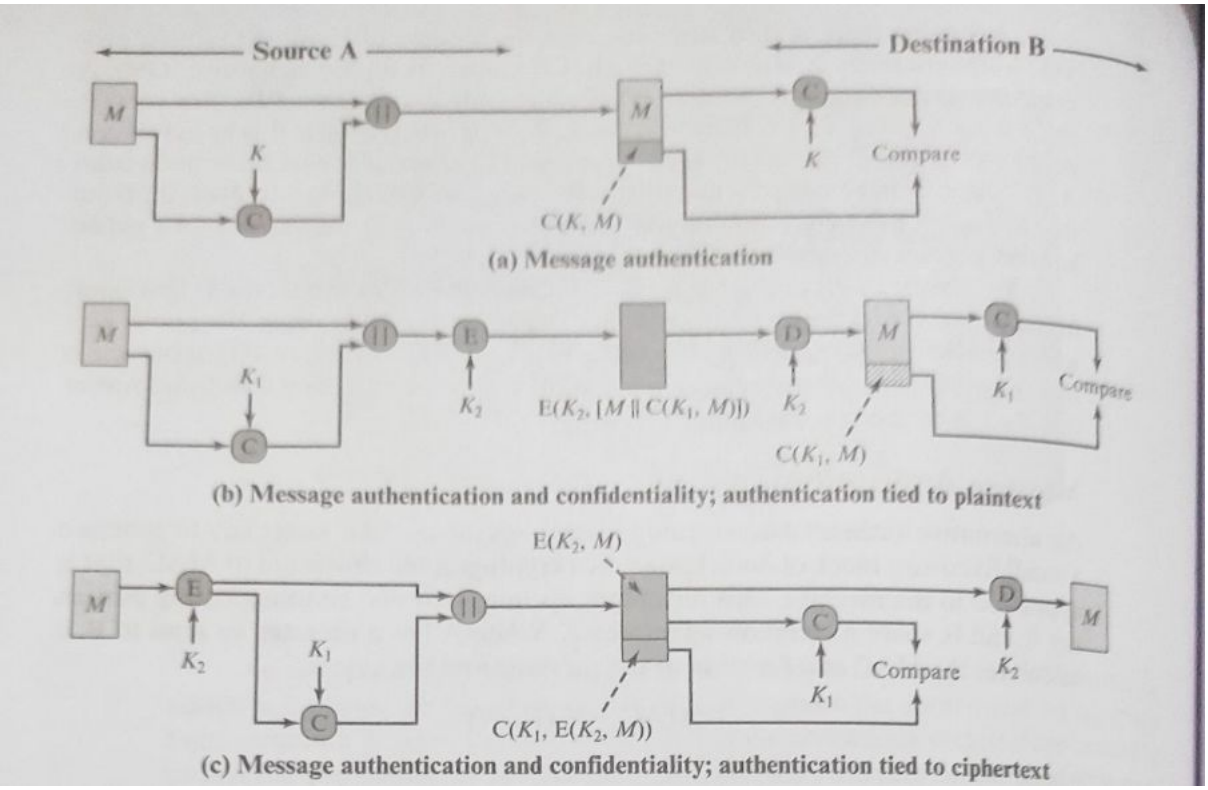
MAC = message authentication code.

The **message plus MAC** are transmitted to the intended recipient.The recipient performs the same calculation on the received message, using the same secret key, to generate a new MAC. The received MAC is then compared to the calculated MAC. If the received MAC matches the calculated MAC, two things can be assured.

1.Assurance that the message has not been altered-If an attacker alters the message but does not alter the MAC, then the receiver's calculation of the MAC will differ from the received MAC. Because the attacker is assumed not to know the secret key, the attacker cannot alter the MAC to correspond to the alterations in the message. Therefore, if the MAC matches, the receiver is assured that the message has not been altered.

2. Assurance that the message is from the alleged sender-Because no one else knows the secret key, no one else could prepare a message with a proper MAC. Therefore, the receiver is assured that the message is from the alleged sender.

Message Authentication Using MAC:



The process shown in Figure (a) is the basic message authentication process.

Explanation

At **Source A**: Message **M** is given as input. The MAC function **C** uses the shared secret key **K**.

MAC is generated as: MAC=C(K,M)

The MAC is combined with the message. The message and MAC are transmitted to **Destination B**.

At **Destination B**: The received message is used as input to the same MAC function. The same secret key **K** is used. A new MAC is calculated. The received MAC is compared with the calculated MAC. If they match, the message is authenticated.

If the message includes a sequence number, such as is used with **HDLC, X.25, and TCP**, then the receiver can be assured of the proper sequence because an attacker cannot successfully alter the sequence number. Thus, when a message contains a sequence number, the authentication process also provides assurance of the **proper sequence**.

MAC Function-A MAC function is similar to encryption. There is one important difference , The MAC algorithm need not be reversible, as it must be for decryption. In general, the MAC function is a: **many-to-one function**. The **domain** of the function consists of messages of some arbitrary length. The **range** consists of all possible MACs.

If an n-bit MAC is used, then there are: {2^n} possible MACs.

There are N possible messages, where: N>> 2^n

That means the number of possible messages is much greater than the number of possible MAC values. Furthermore, with a **k-bit key**, there are: {2^k} possible keys.

Example-100-bit Message and 10-bit MAC:

Suppose we are using:

- 100-bit messages
- 10-bit MAC

There are: 2^{100} different possible messages.

But there are only: 2^{10} different MACs.

Therefore, on average, each MAC value is generated by:

$\frac{2^{100}}{2^{10}}$ different messages.

Therefore: 2^{90} different messages, on average, generate each MAC value.

Example: 5-bit Key

If a **5-bit key** is used, then: 2^5=32

Therefore, there are: 32 different mappings from the set of messages to the set of MAC values.

Because of the **mathematical properties of the authentication function**, it is **less vulnerable to being broken than encryption**.

MAC Provides Authentication but Not Confidentiality-The process shown in Figure provides **Authentication**. but does not provide confidentiality. The reason is that the message is **sent in the clear.** Therefore, confidentiality must be provided separately if it is required.

Providing Confidentiality with MAC-Confidentiality can be provided by performing message encryption either:

1. After the MAC algorithm, or
2. Before the MAC algorithm.

In both cases, two separate keys are needed, each of which is shared by the sender and the receiver.

Message Authentication and Confidentiality — Authentication Tied to Plaintext: This is shown in diagram (b). In the first case, The MAC is calculated using the message as input and is then concatenated to the message. The entire block is then encrypted.

The process is: M

First, calculate: C(K1,M)

The MAC is then concatenated with the message: M || $C(K_1,M)$.

The entire block is then encrypted using K_2: $E(K_2,[M||C(K_1,M)])$.

Message Authentication and Confidentiality — Authentication Tied to Ciphertext: This is shown in **Diagram (c)**. In the second case, The message is encrypted first. Then the MAC is calculated using the resulting ciphertext and is concatenated to the ciphertext to form the transmitted block.

First, the message is encrypted using K_2: $E(K_2,M)$

Then the MAC is calculated using the resulting ciphertext: $C(K_1,E(K_2,M))$.

The MAC is concatenated to the ciphertext.

Typically, it is preferable to tie the authentication directly to the plaintext, so the method of Diagram (b) is used. The reason stated immediately before this is that the message is first used as the input to calculate the MAC and the resulting block is then encrypted.

Because symmetric encryption will provide authentication and because it is widely used with readily available products, why not simply use this instead of a separate message authentication code? given **six situations** in which a separate message authentication code is used.

Situation 1 — Same Message Broadcast to Many Destinations

There are applications in which the **same message is broadcast to a number of destinations**.

Examples given in the attachment are:

- Notification to users that the network is now unavailable

- An alarm signal in a military control center .

It is cheaper and more reliable to have only one destination responsible for monitoring authenticity.

Therefore:

- The message must be broadcast in plaintext.
- An associated message authentication code is used.
- The responsible system has the secret key.
- The responsible system performs authentication.
- If a violation occurs, the other destination systems are alerted by a general alarm.

Situation 2 — Heavy Load at One Side: Another possible scenario is an exchange in which one side has a heavy load and cannot afford the time to decrypt all incoming messages. In this case, Authentication is carried out on a selective basis. Messages are chosen at random for checking.

Situation 3 — Authentication of a Computer Program: Authentication of a **computer program in plaintext** is an attractive service. The computer program can be executed without having to decrypt it every time. Decrypting it every time would be **wasteful of processor resources**. However, if a message authentication code were attached to the program, it could be checked whenever assurance was required of the integrity of the program.

Situation 4 — Authentication Without Keeping Messages Secret: For some applications, it may not be important to keep messages secret, but it is important to authenticate messages. The attachment gives Simple Network Management Protocol Version 3 (SNMPv3) as an example.

SNMPv3 separates the functions of:

- Confidentiality
- Authentication

For this application, it is usually important to authenticate incoming SNMP messages, particularly if the message contains a command to change parameters at the managed system. On the other hand, it may not be necessary to conceal the SNMP traffic.

Situation 5 — Separation of Authentication and Confidentiality: Separation of authentication and confidentiality functions offers architectural flexibility.

For example:

- Authentication may be performed at the application level.
- Confidentiality may be provided at a lower level, such as the transport layer.

Situation 6 — Protection Beyond the Time of Reception: A user may wish to prolong the period of protection beyond the time of reception while still allowing processing of the message contents.

With message encryption:

The protection is lost when the message is decrypted. Therefore, the message is protected against fraudulent modifications only in transit, but not within the target system.

MAC Does Not Provide a Digital Signature: The MAC does not provide a digital signature, because both sender and receiver share the same key. Thus, the MAC uses a **shared secret key** between the sender and receiver.

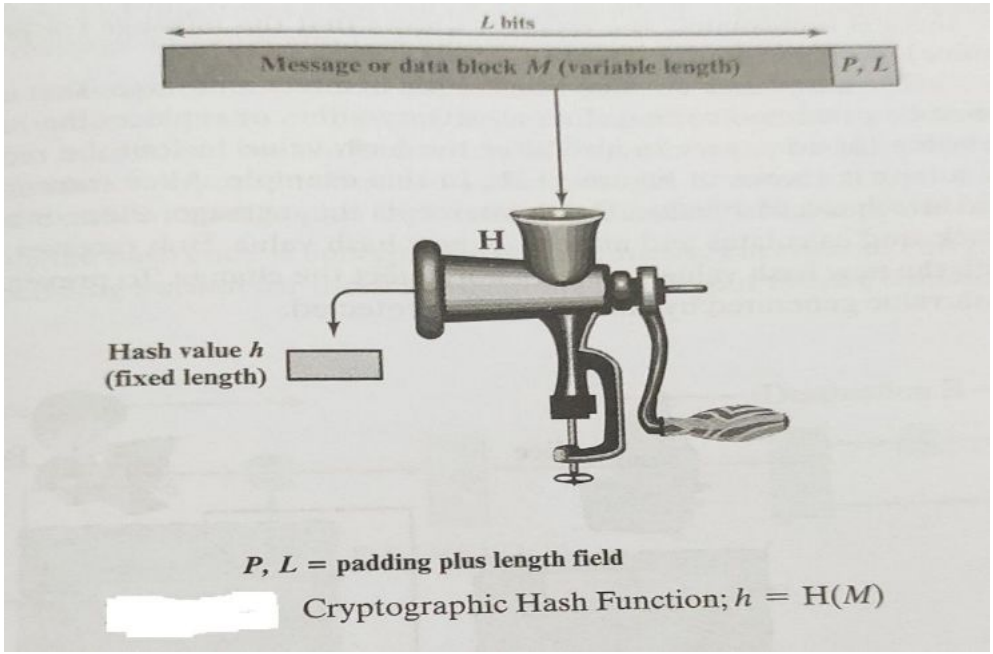
HASH FUNCTION

A hash function H accepts a variable-length block of data as input and produces a fixed-size result: h=H(M)

where:

- M = message/input
- H = hash function
- h = hash value or hash code

A good hash function has the property that the results of applying it to a large set of inputs will produce outputs that are evenly distributed and apparently random. In general terms, the principal objective of a hash function is data integrity. A change to any bit or bits in M results, with high probability, in a change to the hash value.



Typically, the input is padded out to an integer multiple of some fixed length. The padding includes a field containing the length of the message in bits. The length field is a security measure intended to increase the difficulty for an attacker to produce an alternative message with the same hash value.

Cryptographic Hash Function-The kind of hash function needed for security applications is referred to as a cryptographic hash function. A cryptographic hash function is an algorithm for which it is computationally infeasible to find either:

1. A data object that maps to a pre-specified **hash result** — the **one-way property**.
2. Two data objects that map to the **same hash result** — the **collision-free property**.

Because of these characteristics, hash functions are often used to determine whether or not data has changed.

Applications of Cryptographic Hash Functions- used in a wide variety of security applications.

The applications discussed are:

Message Authentication-Message authentication is a mechanism or service used to verify the integrity of a message. Message authentication assures that data received are exactly as sent, with no:

- modification
- insertion
- deletion
- replay

When a hash function is used to provide message authentication, the hash function value is often referred to as a **message digest**.

The basic use of a hash function for message integrity is:

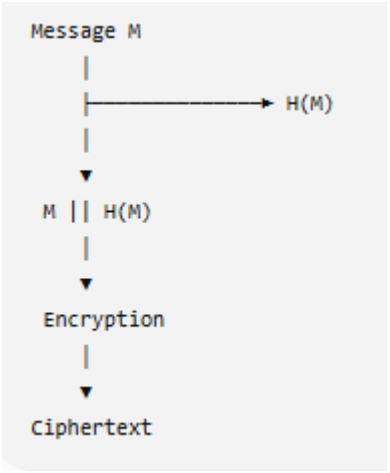
1. The sender computes a hash value as a function of the bits in the message.
2. The sender transmits both the message and the hash value.
3. The receiver performs the same hash calculation on the received message.
4. The receiver compares this value with the incoming hash value.
5. If there is a mismatch, the message or hash value has been altered.

The hash value itself must be protected. Otherwise, an attacker could alter the message and replace the hash value with a newly generated hash value.

Use of Hash Code for Message Authentication

Method 1: Encrypt message + hash code

The message plus the concatenated hash code is encrypted using symmetric encryption.



Because the communicating parties share the secret key, the message must have come from the claimed sender and must not have been altered. The hash code provides the structure for redundancy required to achieve authentication. Because encryption is applied to the entire message plus hash code, confidentiality is also provided.

Method 2: Encrypt only the hash code

Only the hash code is encrypted using symmetric encryption. This reduces the processing requirement for applications that do not require confidentiality.

Method 3: Use a secret value with the hash

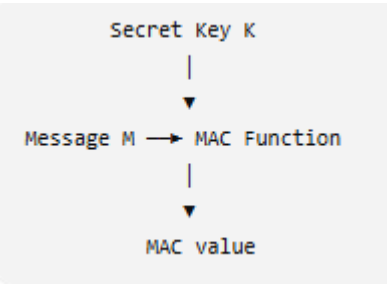
A hash function can be used with a secret value so that the resulting hash value cannot be reproduced by an attacker who does not know the secret.

Message Authentication Code (MAC)

The message authentication is more commonly achieved using a **message authentication code (MAC)**, also known as a **keyed hash function**. A MAC is used between two parties that share a secret key. A MAC function takes as input:

- a secret key
- a data block

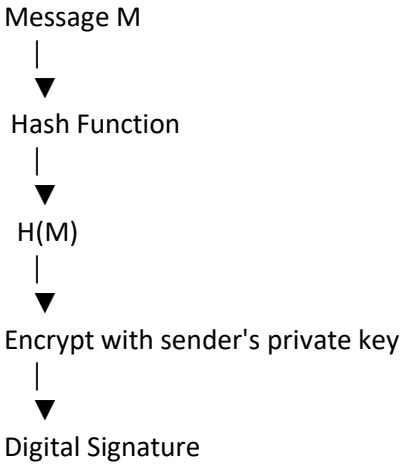
and produces a fixed-size hash value called the **MAC**, which is associated with the protected message.



If the integrity of the message needs to be checked, the MAC function is applied to the message and the result is compared with the associated MAC value. An attacker who alters the message will be unable to alter the associated MAC value without knowledge of the secret key.

Digital Signatures

The operation of the digital signature is similar to that of the MAC .The hash value of the message is encrypted using the sender's **private key**. Anyone who knows the user's public key can verify the integrity of the message associated with the digital signature.



The two cases:

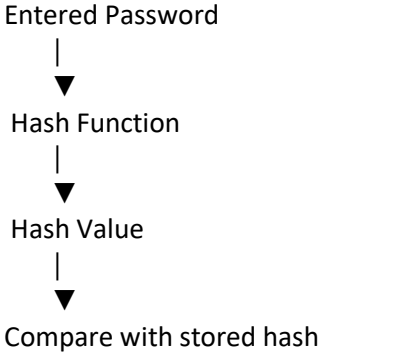
a. The hash code is encrypted using public-key encryption with the sender's private key. This provides authentication and a digital signature.

b. If confidentiality as well as a digital signature is desired, the message plus the private-key-encrypted hash code can be encrypted using a symmetric secret key.

Other Applications

Password Protection- Hash functions are commonly used to create a one-way password file. A hash of a password is stored by an operating system rather than the password itself. Thus, the actual password is not retrievable by a user who gains access to the password file.

When a user enters a password:



The hash of the entered password is compared with the stored hash value for verification.

Intrusion Detection- Hash functions can be used for intrusion detection. A hash value can be generated for each file on a system.

For example: $H(F)$

where **F** represents a file.

The hash values are kept securely. Later, the hash value can be determined again to check whether the file has changed. If the file has been changed, its hash value will normally change.

Virus Detection-Hash Functions are also used in Virus detection.

Two Simple Hash Functions

The two simple hash functions to provide an understanding of the security considerations involved in cryptographic hash functions. The input message is viewed as a sequence of **n-bit blocks**. The input is processed one block at a time in an iterative fashion to produce an **n-bit hash**.

Simple XOR Hash Function-One of the simplest hash functions is the **bit-by-bit exclusive-OR (XOR)** of every block. It can be expressed as:

$$Ci=b1i \oplus b2i \oplus \cdots \oplus bmi$$

where:

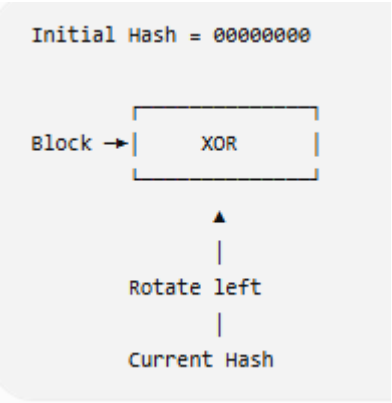
- Ci= ith bit of the hash code
- i=1 to n
- m = number of n-bit blocks in the input
- bij= ith bit in the jth block

This operation produces a simple parity bit for each bit position and is known as a **longitudinal redundancy check**. It is reasonably effective for random data as a data integrity check.

Rotated XOR (RXOR)- A simple way to improve the XOR method is to perform a **one-bit circular shift or rotation** on the hash value after each block is processed.

Procedure

1. Initially set the n-bit hash value to zero.
2. Process each successive n-bit block of data:
    - Rotate the current hash value to the left by one bit.
    - XOR the block into the hash value.



This has the effect of **randomizing the input more completely** and overcoming regularities that appear in the input.

Security Limitation of Simple Hash Functions- Although the second procedure provides a good measure of data integrity, it is **virtually useless for data security** when an encrypted hash code is used with a plaintext message. Given a message, it is an easy matter to produce a new message that yields the same hash code. An attacker can prepare an alternate message and append a block that forces the new message plus the block to yield the desired hash code. Therefore, simple XOR or rotated XOR is not adequate for cryptographic security.

Preimage and Collision

For a hash value: $h=H(x)$

x is called the **preimage** of **h**.

A **collision** occurs when: $x \neq y$

But $H(x)=H(y)$

Because hash functions are used for data integrity, collisions are undesirable.

Security Requirements for Cryptographic Hash Functions

1. Variable-length input-The hash function should be applicable to a block of data of any length.

2. Fixed-length output-The hash function should produce a fixed-length output**.**

3. Ease of computation-The function should be relatively easy to compute for any given data object, making both hardware and software implementations practical.

4. Preimage resistance-For any given hash value h, it should be computationally infeasible to find y such that: $H(y)=h$ This is the one-way property. It is easy to generate a hash code given a message, but virtually impossible to generate a message given a hash code.

5. Second preimage resistance-For any given block x, it should be computationally infeasible to find: $y \neq x$

such that: $H(y)=H(x)$

This guarantees that it is infeasible to find an alternative message with the same hash value as a given message.

Collision resistance-It should be computationally infeasible to find any pair: $(x,y),x\neq y$

such that: $H(x)=H(y)$

Pseudorandomness-The output of **H** should meet standard tests for pseudorandomness.

Preimage Resistance-reimage resistance is the **one-way property**.It is easy to generate a hash code given a message, but virtually impossible to generate a message given a hash code. This property is important when authentication involves a secret value. If the hash function were not one-way, an attacker could observe or intercept a transmission containing: M and: $h=H(S||M)$

The attacker could then invert the hash function to obtain: $S||M$ and recover the secret value **S**.

Second Preimage Resistance-Second preimage resistance guarantees that it is infeasible to find an alternative message with the same hash value as a given message.This prevents forgery when an encrypted hash code is used.

Without this property, an attacker could:

1. Observe or intercept a message and its encrypted hash code.
2. Generate an unencrypted hash code from the message.
3. Generate an alternate message with the same hash code.

Collision Resistance-A hash function is collision resistant if it is computationally infeasible to find two messages that have the same hash value.

It distinguishes the terms:

- Preimage resistant
- Second preimage resistant
- Collision resistant

A collision-resistant function is also second- preimage resistant, but the reverse is not necessarily true.

Birthday Attack- The **birthday paradox** in relation to collision resistance. For a hash code of length **n bits**, finding two data blocks having the same hash value requires considerably less effort than a preimage attack. For a uniformly distributed hash function, two data blocks can be expected to have the same hash value within approximately: $\{2\text{^}\{n/2\}\}$ attempts.This is known as the **birthday attack**.

Iterated Hash Function Structure

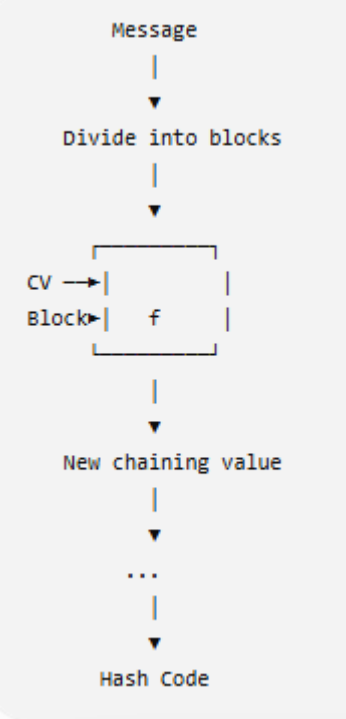
The hash function:

1. Takes an input message.
2. Partitions it into fixed-size blocks.
3. Pads the final block if necessary.
4. Includes the total length of the input.
5. Processes the blocks using a compression function.
6. Produces the final hash code.

The compression function takes two inputs:

- an **n-bit chaining variable**
- a **b-bit message block**

and produces an **n-bit output**.



The structure can be used to produce a secure hash function.

The design of a secure hash function is closely related to designing a **collision-resistant compression function**.

Security of Hash Functions

Preimage and Collision-For a hash value: $h=H(x)$,**x** is called the **preimage** of **h**. Because a hash function is a many-to-one mapping, for a given hash value **h**, there will in general be multiple preimages.

A **collision** occurs when: $x \neq y$ and $H(x)=H(y)$

Since hash functions are used for data integrity, **collisions are clearly undesirable**.
Security Requirements for Cryptographic Hash Functions

Requirement	Description
Variable input size	It can be applied to a block of data of any size.
Fixed output size	It produces a fixed-length output.
Efficiency	H(x)H(x) is relatively easy to compute for any given xx, making both hardware and software implementations practical.
Preimage resistant (one-way property)	For any given hash value hh, it is computationally infeasible to find xx such that H(x)=hH(x)=h.
Second preimage resistant (weak collision resistance)	For any given block xx, it is computationally infeasible to find y≠xy\neq x such that H(y)=H(x)H(y)=H(x).
Collision resistant (strong collision resistant)	It is computationally infeasible to find any pair (x,y)(x,y), with x≠yx\neq y, such that H(x)=H(y)H(x)=H(y).
Pseudorandomness	The output of HH meets standard tests for pseudorandomness.

Preimage Resistance- The preimage resistant property is the one-way property. It is easy to generate a hash code given a message, but it should be virtually impossible to generate a message given a hash code. This property is important when the authentication technique involves the use of a secret value. If the hash function is not one-way, an attacker who observes or intercepts a transmission can potentially discover the secret value by inverting the hash function.

Second Preimage Resistance-The second preimage resistant property guarantees that it is infeasible to find an alternative message with the same hash value as a given message.This property prevents forgery when an encrypted hash code is used.

If this property did not hold, an attacker could:

1. Observe or intercept a message together with its encrypted hash code.
2. Generate an unencrypted hash code from the message.
3. Generate an alternative message having the same hash code.

Collision Resistance- For a collision-resistant attack, an adversary attempts to find two messages or data blocks xx and yy that produce the same hash function value: H(x)=H(y), This requires considerably less effort than a preimage or second-preimage attack. The effort required is related to the birthday paradox. For an m-bit hash value, two data blocks with the same hash value can be expected to be found within approximately:{2^{m/2}} attempts.

Brute-Force Attacks-The two categories of attacks on hash functions:

1. **Brute-force attacks**
2. **Cryptanalysis**

A brute-force attack does not depend on the specific algorithm but depends on the **bit length**.For a hash function, a brute-force attack depends only on the **bit length of the hash value**. Preimage / Second Preimage Attack. For an m-bit hash value, the brute-force method attempts different values until a value is found that produces the required hash. The average effort is approximately:{2^m}

Collision Attack-For an m-bit hash value:{2^{m/2}} attempts are required on average to find a collision.

Cryptanalysis-Cryptanalytic attacks on hash functions attempt to exploit some property of the algorithm to perform an attack other than an exhaustive search.The strength of a hash algorithm against cryptanalysis can be measured by comparing its resistance to the effort required for a **brute-force attack**. An ideal hash algorithm requires cryptanalytic effort greater than or equal to the brute-force effort.

SECURE HASH ALGORITHM (SHA)

The **Secure Hash Algorithm (SHA)** is one of the most widely used hash functions. SHA was developed by the **National Institute of Standards and Technology (NIST)** and published as a **Federal Information Processing Standard (FIPS 180)**. The original standard was revised, resulting in **SHA-1**.

Later, NIST published **FIPS 180-2**, which defined three new versions:

- **SHA-256**
- **SHA-384**
- **SHA-512**

These are collectively known as **SHA-2**. A later revision added **SHA-224**.

SHA-1- SHA-1 produces a: 160-bit hash value. A simple brute-force technique for finding two different messages that produce the same hash value would require, on average:{2^{80}} .SHA-1 was subsequently found to have practical collision attacks. The development of SHA-2 as the replacement family and discusses the later practical collision result for SHA-1.

SHA-2-The SHA-2 family includes:

Algorithm	Hash value
SHA-224	224 bits
SHA-256	256 bits
SHA-384	384 bits
SHA-512	512 bits

The new versions have the same underlying general structure and use the same types of modular arithmetic and logical binary operations as SHA-1.

SHA-512

Basic Characteristics

SHA-512:

- takes a message of less than 21282^{128} bits;
- processes the message in **1024-bit blocks**;
- produces a **512-bit message digest**.

The processing follows the general iterative structure of a cryptographic hash function.

SHA-512 Processing

The SHA-512 algorithm consists of the following major steps:

1. Append padding bits.
2. Append the length.
3. Initialize the hash buffer.
4. Process the message in 1024-bit blocks.
5. Produce the final message digest.

Step 1: Append Padding Bits

The message is padded so that its length becomes: $896 (\mathrm{mod} 1024)$

Padding is always performed, even if the message is already of the desired length.

The padding consists of:

- a single **1 bit**
- followed by the required number of **0 bits**.

The number of padding bits is between **1 and 1024**.

Step 2: Append Length

A block of: 128 bits is appended to the message. This block contains the length of the **original message in bits**, before padding. It is treated as an unsigned 128-bit integer, with the most significant byte first. After these first two steps, the resulting message has a length that is an integer multiple of: 1024 bits. The expanded message consists of 1024-bit blocks: $M_1, M_2, \ldots, M_N$

Step 3: Initialize the Hash Buffer

A **512-bit buffer** is used to hold intermediate and final results. The buffer consists of eight 64-bit registers: a, b, c, d, e, f, g, h

The initial values are:

H0=6A09E667F3BCC908

H1=BB67AE8584CAA73B

H2=3C6EF372FE94F82B

H3=A54FF53A5F1D36F1

H4=510E527FADE682D1

H5=9B05688C2B3E6C1F

H6=1F83D9ABFB41BD6B

H7=5BE0CD19137E2179

These values are obtained from the fractional parts of the square roots of the first eight prime numbers.

Step 4: Process Each 1024-Bit Block

The heart of SHA-512 is the processing of each 1024-bit message block. Each block is processed through **80 rounds**.

Each round uses:

- a 64-bit word WtW_t
- a 64-bit additive constant KtK_t
- the eight working variables a,b,c,d,e,f,g,ha,b,c,d,e,f,g,h

The constants represent the first 64 bits of the fractional parts of the cube roots of the first 80 prime numbers.

Message Schedule-Each 1024-bit block is divided into **sixteen 64-bit words**. The first 16 words are taken directly from the current message block. The remaining words are generated using:

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}$$

The additions are performed modulo:2^{64}

The functions σ0 and σ1 use rotations and shifts.

SHA-512 Round Function

For each of the 80 rounds, the following operations are performed.

$$T_1 = h + Ch(e,f,g) + \Sigma_1(e) + W_t + K_t$$

$$T_2 = \Sigma_0(a) + Maj(a,b,c)$$

Then the working variables are updated:

h=g , g=f, f=e, e=d+T1, d=c, c=b, b=a, a=T1+T2

Functions Used in SHA-512

Ch Function

Ch(e,f,g)=(e AND f)⊕(e⁻ AND g)

It is the **conditional function**: if e, then f, else g.

Maj Function

Maj(a,b,c)=(a AND b)⊕(a AND c)⊕(b AND c)

The function is true when the **majority of the three arguments are true**.

Σ_0

$$\Sigma_0(a) = ROTR^{28}(a) \oplus ROTR^{34}(a) \oplus ROTR^{39}(a)$$

Σ_1

$$\Sigma_1(e) = ROTR^{14}(e) \oplus ROTR^{18}(e) \oplus ROTR^{41}(e)$$

where $ROTR^n(x)$ represents a circular right shift of the 64-bit argument by n bits.

Final Hash Computation

After the 80th round, the output is added to the input of the first round.The addition is performed independently for each of the eight words using addition modulo:2^{64}

After all N blocks have been processed, the output from the final stage is the:

512-bit message digest

$$H_i = SUM_{64}(H_{i-1}, a, b, c, d, e, f, g, h)$$

and the final message digest is obtained from the final hash value.

SHA-512 Example

The example using the three-character ASCII message: "abc"

The ASCII representation is:

a = 01100001
b = 01100010
c = 01100011

The original message length is: 3×8=24 bits

The message is padded to 896 bits modulo 1024, followed by the 128-bit length field.

Thus, the resulting message block is exactly: 1024 bits

Security of SHA-512

The PDF explains that the SHA-512 processing produces a well-mixed output.

Every bit of the input has an effect on the output, and the result appears random even when the input messages have similar characteristics.

The difficulty of finding two messages having the same hash value is approximately related to the **birthday attack**, while finding a message corresponding to a given digest requires considerably greater effort.

This also demonstrates the **avalanche effect**: changing the input results in a substantial change in the resulting hash value.

SHA-3

SHA-3 uses a different basic structure from SHA-1 and SHA-2. The overall SHA-3 function is a **sponge function**. It can be expressed as: $\mathrm{Keccak}(r,c)$

where:

- r = **rate**
- c = **capacity**

and: $r+c=1600$ for the SHA-3 state. The SHA-3 processing uses the **Keccak-f iteration function**.

SHA-3 State Matrix-The SHA-3 state is represented using a matrix of **64-bit words**. The Keccak-f function operates on this state through a sequence of transformations.

The operations include:

- Boolean operations
- rotations
- bit manipulation

There is no need for table lookups, arithmetic operations, or data-dependent rotations for these operations, making SHA-3 suitable for efficient hardware and software implementation.

HMAC — Hash-Based Message Authentication Code

HMAC stands for Hash-based Message Authentication Code. HMAC is a MAC based on a cryptographic hash function. Cryptographic hash functions such as MD5 and SHA generally execute faster in software than symmetric block ciphers such as DES, and library code for hash functions is widely available. A hash function such as SHA cannot be used directly as a MAC, because it does not use a secret key. HMAC incorporates a secret key into an existing hash algorithm. HMAC has been specified in RFC 2104 and FIPS 198.

HMAC Design Objectives

The design objectives of HMAC are:

1. **To use, without modifications, available hash functions.**
    - In particular, hash functions that perform well in software and whose code is freely and widely available.

2. **To allow easy replacement of the embedded hash function**
    - This is useful when a faster or more secure hash function is available.
3. **To preserve the original performance of the hash function**
    - Without significant degradation.
4. **To use and handle keys in a simple way.**
5. **To have a well-understood cryptographic analysis of the strength of the authentication mechanism**
    - Based on reasonable assumptions about the embedded hash function.

HMAC treats the hash function as a **"black box."**

Therefore:

- An existing implementation of a hash function can be used in HMAC without modification.
- The hash function can be replaced if a faster or more secure hash function is required.

HMAC Structure- The HMAC algorithm uses the following parameters**:**

- **H** = embedded hash function
- **M** = message
- **K** = secret key
- **K^+** = key padded with zeros to the block size
- **ipad** = inner padding
- **opad** = outer padding

ipad = 00110110 (36 hexadecimal) repeated b/8 times

opad = 01011100 (5C hexadecimal) repeated b/8 times.

HMAC Formula

$$HMAC(K, M) = H[(K^+ \oplus opad) \,||\, H[(K^+ \oplus ipad) \,||\, M]]$$

Where:

- K = secret key
- M = message
- H = hash function
- K^+ = padded key
- $\oplus$ = XOR
- $||$ = concatenation

HMAC Algorithm

The algorithm can be explained in the following steps.

Step 1: Prepare the key

Append zeros to the left end of the key **K** to create a **b-bit string** K^+.

If the key is longer than the block size, it is first processed by the hash function to produce an appropriate key.

Step 2: Inner XOR

XOR K^+ with **ipad**.

$$S_i = K^+ \oplus ipad$$

Step 3: Append the message

Append the message **M** to S_i.

$$S_i||M$$

Step 4: Apply the hash function

Calculate:

$$H(S_i||M)$$

This produces the **inner hash result.**

Step 5: Outer XOR

XOR K^+ with **opad**.

$$S_o = K^+ \oplus opad$$

Step 6: Append inner hash

Append the inner hash result to S_o.

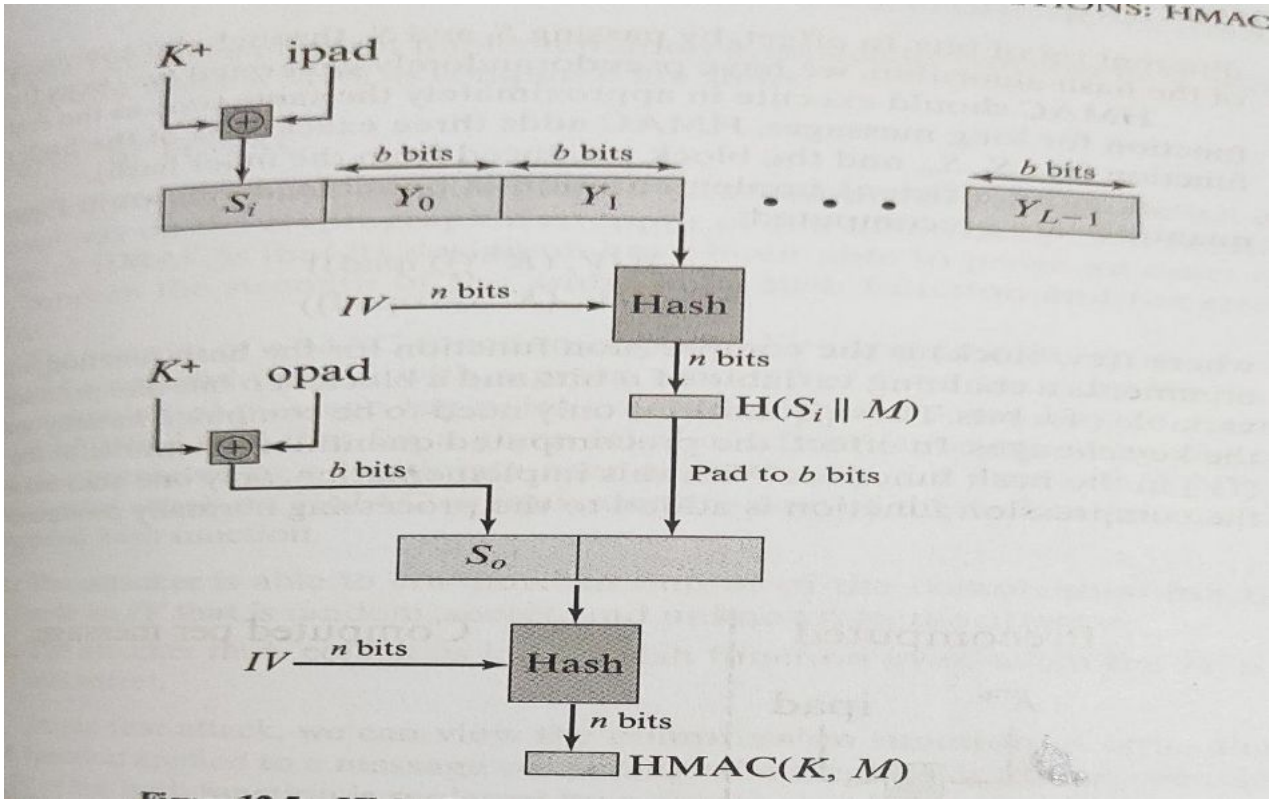
$$S_o||H(S_i||M)$$

Step 7: Apply the hash function again

Apply the hash function to obtain the final HMAC:

$$\boxed{H(S_o||H(S_i||M))}$$

HMAC Diagram



Performance of HMAC

HMAC should execute in approximately the same time as the embedded hash function for long messages. HMAC adds three executions of the hash compression function:

1. For Si
2. For So
3. For the block produced from the inner hash

A more efficient implementation is possible by **precomputing** the quantities associated with the inner and outer pads

---

CMAC — Cipher-Based Message Authentication Code

CMAC stands for **Cipher-based Message Authentication Code**. It is a **MAC based on a block cipher**. The Data Authentication Algorithm, also known as **CBC-MAC**, had a limitation, it was secure under a restriction that only messages of one fixed length were processed. This limitation was overcome by a construction using additional keys. The refined construction was adopted by NIST as **CMAC**. CMAC is specified in **NIST Special Publication 800-38B** and is used with **AES and triple DES**.

CMAC with AES and Triple DES

For **AES**:

- Block size = **128 bits**
- Key size = **128, 192, or 256 bits**

For **Triple DES**:

- Block size = **64 bits**
- Key size = **112 or 168 bits**

The message is divided into blocks of the cipher block length.

CMAC Operation- For a message that is an integer multiple of the cipher block length:

$$C_1 = E(K, M_1)$$

$$C_2 = E(K, M_2 \oplus C_1)$$

$$C_3 = E(K, M_3 \oplus C_2)$$

The final block is processed as:

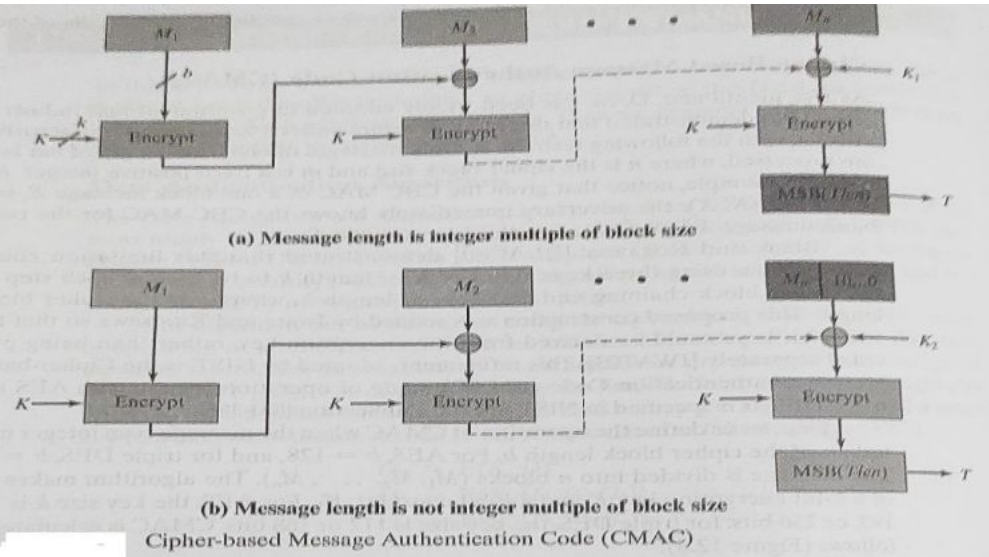
$$C_n = E(K, [M_n \oplus C_{n-1} \oplus K_1])$$

The authentication tag is obtained as:

$$T = MSB_{Tlen}(C_n)$$

where **T** is the message authentication code, also called the **tag**.

CMAC Diagram:



(a) Message length is integer multiple of block size

(b) Message length is not integer multiple of block size

Cipher-based Message Authentication Code (CMAC)

When the Message Is Not a Complete Block

If the message is **not an integer multiple of the cipher block length**, the final block is padded on the right with: **1 followed by as many 0s as necessary.**

so that the final block becomes the required block length.

For this case, the CMAC operation proceeds as before, but K_2 is used instead of K_1.

Generation of K_1 and K_2

The two additional b-bit keys are derived from the encryption key.

First:

$$L = E(K, 0^b)$$

Then:

$$K_1 = L \cdot x$$

and

$$K_2 = L \cdot x^2$$

The multiplication is performed in the finite field:

$$GF(2^b)$$

It describes x and x^2 as first- and second-order polynomials that are elements of GF(2^b).

HMAC vs CMAC

Feature	HMAC	CMAC
Full form	Hash-based Message Authentication Code	Cipher-based Message Authentication Code
Based on	Cryptographic hash function	Block cipher
Main construction	Hash function + secret key	Block cipher + secret key
Examples mentioned in PDF	MD5, SHA, RIPEMD-160	AES, Triple DES
Padding components	ipad and opad	K1K_1 and K2K_2
Main operation	Inner hash + outer hash	Cipher block chaining operation
Standard mentioned	RFC 2104, FIPS 198	NIST SP 800-38B

Whirlpool

Whirlpool **is a cryptographic hash function that produces a** 512-bit message digest**.** The algorithm takes a message of maximum length less than 2^{256} bits as input and processes the input in 512-bit blocks. **Whirlpool uses a** block-cipher-based hash structure**.** The basic idea is to use an internal block cipher to process each message block and then combine the result with the previous hash value and the current message block.

Whirlpool Hash Structure- Whirlpool uses the general iterated hash structure proposed by Merkle, which is used in virtually all secure hash functions**.**

In Whirlpool:

- The **hash code length** is 512 bits.
- The **cipher block length** is 512 bits.
- The input message is processed in **512-bit blocks**.
- The internal block cipher is called **W**.

For a message consisting of blocks:

$m_1, m_2, \ldots, m_t$

the Whirlpool hash function is expressed as: $H_0 = \text{initial}$

$H_i = H(H_{i-1}, m_i) \oplus H_{i-1} \oplus m_i$

where:

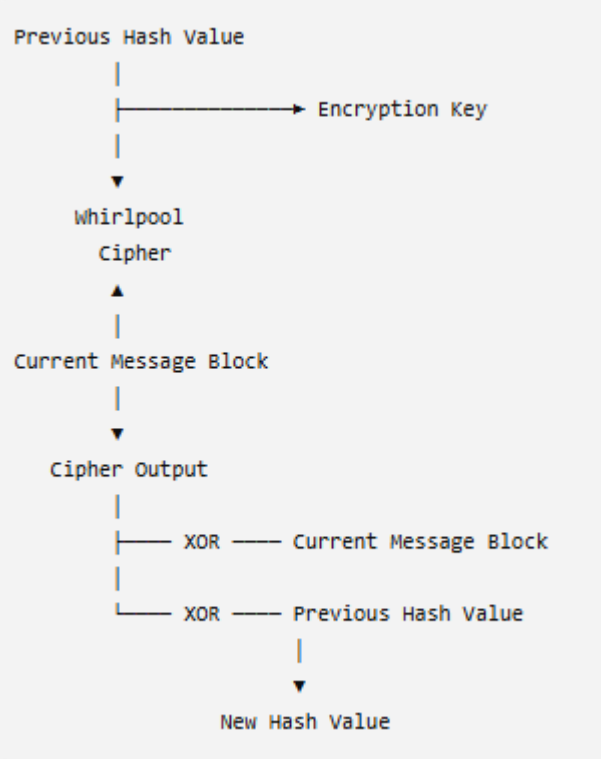
- H_0 = initial value
- m_i = current message block
- H_{i-1} = intermediate hash value from the previous iteration
- H_i= intermediate hash value
- H_t = final hash-code value

Feed-Forward Operation-The Whirlpool compression function uses a feed-forward operation.

For each iteration:

- The **intermediate hash value from the previous iteration** is used as the encryption key input.
- The **current message block** is used as the plaintext input.
- The feed-forward value is the **bitwise XOR of the current message block and the previous intermediate hash value**.

Therefore:



Whirlpool Parameters

Parameter	Whirlpool
Maximum message length	Less than 22562^{256} bits
Input block size	512 bits
Hash/digest size	512 bits
Basic structure	Iterated hash
Compression function	Based on Whirlpool block cipher
Feed-forward	XOR

These parameters are given in the attached material.

Whirlpool Cipher-The basic building block of Whirlpool is a **block cipher**. The Whirlpool cipher uses the same functional building blocks as **AES** and has the same overall structure. Because of this, similar performance and space characteristics can be expected.

The cipher processes its data using an internal **8 × 8 matrix of bytes**, called **CState**.

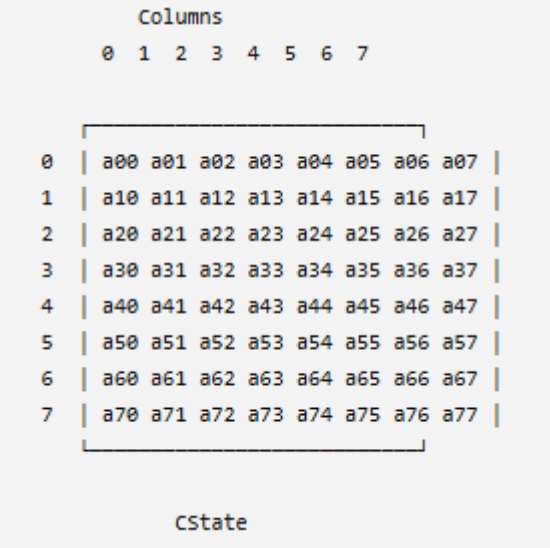
Since: $8 \times 8 = 64$ bytes8

And $64 \times 8 = 512$ bits

the CState represents a **512-bit block**.

Whirlpool Matrix Structure-The input string of bytes is arranged into an **8 × 8 matrix**.

It can be represented as:



Whirlpool Matrix Structure.

Functions of Whirlpool Cipher-The individual functions of the Whirlpool cipher are applied to the internal **CState**. This describes the **Substitute Byte (SB)** function as a nonlinear layer.

Substitute Byte — SB

The substitute byte function is a **simple table lookup** that provides a nonlinear mapping.

Whirlpool defines a **16 × 16 matrix of byte values**, called an **S-box**.

The S-box contains a permutation of all possible **256 8-bit values**.

For each byte:

- The **leftmost 4 bits** are used as the row value.
- The **rightmost 4 bits** are used as the column value.
- The row and column identify the corresponding S-box entry.

- The selected value becomes the output byte.

Example:

For the hexadecimal value:

$$\{95\}$$

the:

- Row = **9**
- Column = **5**

The S-box entry at row 9, column 5 is:

$$\{BA\}$$

Therefore:

$$\boxed{\{95\} \rightarrow \{BA\}}$$

S-Box Structure

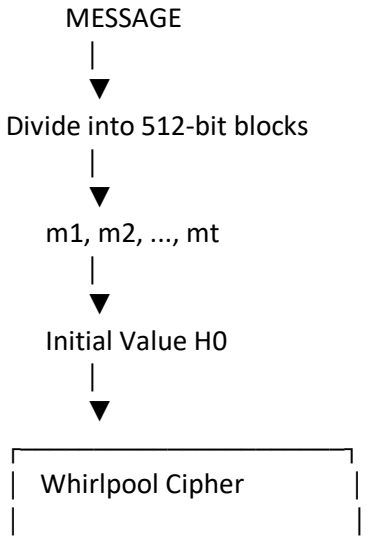
It consists of:

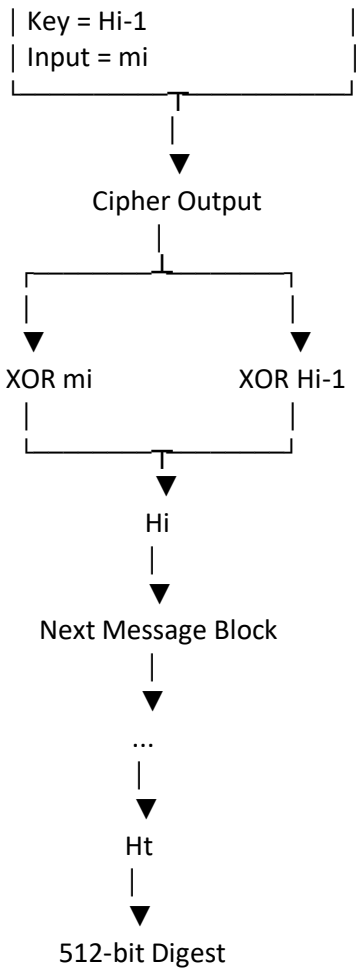
- Two nonlinear layers
- Each layer contains two **4 × 4 S-boxes**
- These are separated by a **4 × 4 randomly generated box**

This provides the nonlinear mapping required by the cipher.

Overall Whirlpool Hashing Process

The complete process can be represented as:





This represents the iterated hash structure and feed-forward operation described .

Security Goals of Whirlpool-The designers of Whirlpool specified security goals including:

1. Collision resistance

The expected workload for generating a collision is of the order of: $2^{n/2}$

executions of Whirlpool, where n is the hash length.

2. Preimage resistance

Given an n-bit value, the expected workload for finding a message that hashes to that value is of the order of: 2^n executions of Whirlpool.

3. Second-preimage resistance

Given a message and its n-bit hash result, the expected workload for finding a second message having the same hash result is of the order of: 2^n executions of Whirlpool.

4. Resistance against linear and differential attacks

It should be infeasible to detect systematic correlations between:

- linear combinations of input bits, and
- linear combinations of bits of the hash result.

It should also be infeasible to predict which bits of the hash result will change when particular input bits are changed.

Whirlpool and AES-Whirlpool uses the **same functional building blocks as AES** and has the **same structure**. Therefore, the this states that similar performance and space characteristics can be expected. However, Whirlpool is being used as a **hash function**, whereas AES is a **symmetric encryption algorithm**.

Performance-A study compared Whirlpool with other secure hash functions, including the versions of SHA. The study concluded that, compared with **SHA-512**:

- Whirlpool requires **more hardware resources**.
- Whirlpool performs **much better in terms of throughput**.

DIGITAL SIGNATURE

The **digital signature** is an important development from public-key cryptography. It provides a set of security capabilities that would be difficult to implement in other ways. Digital signatures are used when the sender wants the receiver to be certain that:

- the message is actually from the claimed sender,
- the contents have not been altered, and
- the signature can be verified by a third party to resolve disputes.

Need for Digital Signature-Message authentication protects two parties who exchange messages from a third party. However, it does not protect the two parties against each other.

Two important disputes can occur:

1. Receiver forges a message

For example, Mary may forge a different message and claim that it came from John.

2. Sender denies sending the message

John may deny sending a message because the authentication mechanism does not provide a way to prove that John actually sent it.

Therefore, when there is not complete trust between sender and receiver, something more than authentication is required.

The digital signature provides a solution to this problem.

Properties of Digital Signature

A digital signature must have the following properties:

1. **Verify the author and the date and time of the signature.**
2. **Authenticate the contents** at the time of the signature.
3. **Be verifiable by third parties** to resolve disputes.

Thus, the digital signature function includes the **authentication function**.

Basic Digital Signature Process

Suppose **Bob wants to send a message to Alice**.

The digital signature process uses:

- Message
- Cryptographic hash function
- Digital signature generation algorithm
- Bob's private key
- Digital signature verification algorithm
- Bob's public key

Digital Signature Process:

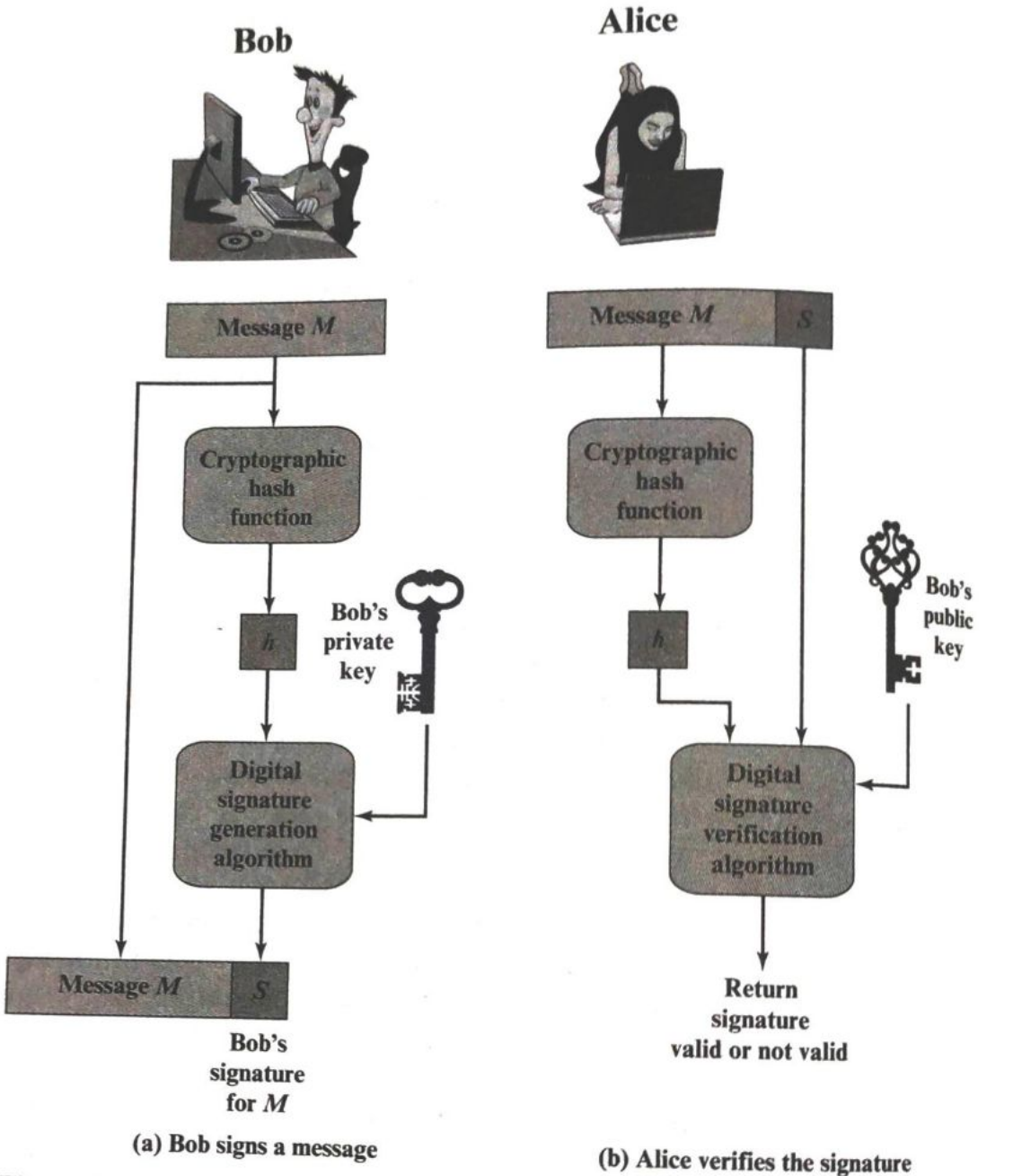


Figure 13.1 Simplified Depiction of Essential Elements of Digital Signature Process

This is the simplified model of the essential elements of the digital signature process

Digital Signature Generation The process at the sender side is:

Step 1: Message-Bob prepares the message M that he wants to send to Alice.

Step 2: Hash the message- Bob uses a secure cryptographic hash function, such as SHA-512, to generate a hash value for the message.

Message M
↓

Cryptographic Hash Function
↓
Hash value h

Step 3: Generate signature-The hash value together with Bob's private key is provided as input to the digital signature generation algorithm.

Hash value h + Bob's Private Key
↓
Digital Signature Generation Algorithm
↓
Digital Signature S

Step 4: Send- Bob sends: Message M + Digital Signature S to Alice.

Digital Signature Verification- When Alice receives the message and signature**:**

Step 1: Calculate hash

Alice calculates the hash value of the received message.

Message M
↓
Cryptographic Hash Function
↓
Hash value h

Step 2: Verify the signature

Alice provides:

- the calculated hash value, and
- Bob's **public key**

as inputs to the digital signature verification algorithm.

Hash value + Bob's Public Key
↓
Digital Signature Verification Algorithm
↓
Valid / Not Valid

If the algorithm returns **valid**, Alice is assured that the message was signed by Bob.

Why Bob's Private Key and Public Key Are Used-

Bob's **private key** is used during signature generation. Bob's **public key** is used during signature verification. Because no one else has Bob's private key, no one else should be able to create a signature that can be verified for this message using Bob's public key. Also, altering the message without access to Bob's private key would cause the verification process to fail.

Therefore, the message is authenticated both in terms of:

- **Source**
- **Data integrity**

Attacks and Forgeries -The PDF identifies several types of attacks against a digital signature scheme, listed in increasing severity.

Let:

- **A** = user whose signature method is being attacked
- **C** = attacker

- Key-Only Attack-The attacker C only knows A's public key.
- Known Message Attack- The attacker is given access to a set of messages and their corresponding signatures.
- Generic Chosen Message Attack-The attacker chooses a list of messages before attempting to break the signature scheme.The messages are chosen independently of A's public key.The attacker then obtains valid signatures for those chosen messages.
- Directed Chosen Message Attack-Similar to the generic chosen message attack, except that the attacker chooses the list of messages after knowing A's public key, but before seeing any signatures.
- Adaptive Chosen Message Attack-The attacker is allowed to use A as an "oracle." The attacker can request signatures for messages that depend on previously obtained message-signature pairs.

Types of Successful Forgeries

Breaking a signature scheme can result in the following:

1. Total Break- The attacker determines the user's private key.

2. Universal Forgery-The attacker finds an efficient signing algorithm that provides an equivalent way of constructing signatures on arbitrary messages.

3. Selective Forgery-The attacker forges a signature for a particular message chosen by the attacker.

4. Existential Forgery-The attacker forges a signature for at least one message, but has no control over which message.

Digital Signature Requirements

A digital signature should satisfy the following requirements:

1. The signature must be a bit pattern that depends on the message being signed.
2. The signature must use some information known only to the sender to prevent forgery and denial.

3. It must be relatively easy to produce the digital signature.
4. It must be relatively easy to recognize and verify the digital signature.
5. It must be computationally infeasible to forge a digital signature:
    - by constructing a new message for an existing digital signature, or
    - by constructing a fraudulent digital signature for a given message.
6. It must be practical to retain a copy of the digital signatur**e in storage**.

A secure hash function embedded in a digital signature scheme provides a basis for satisfying these requirements.

Direct Digital Signature-The term **direct digital signature** refers to a digital signature scheme that involves only the communicating parties: Source → Destination. No third party is involved in the direct digital signature scheme.

Digital signature is a mechanism that provides authentication of the message contents and enables verification of the author and signature by using a digital signature generation and verification process.

DIGITAL SIGNATURE – AUTHENTICATION PROTOCOLS

1. Authentication Protocols-The basic cryptographic tools are used in many applications, including digital signatures.

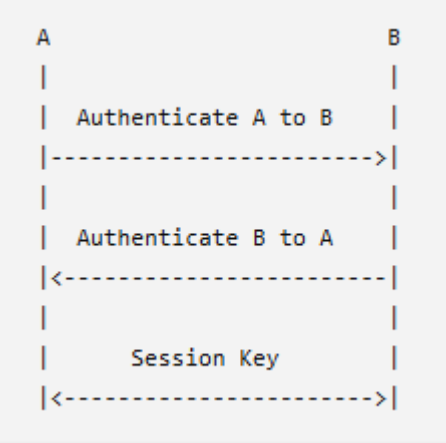
Authentication protocols mainly focus on two areas:

1. **Mutual authentication**
2. **One-way authentication**

These protocols are concerned with establishing the identity of communicating parties and protecting communication against attacks such as message replay.

Mutual AuthenticationMutual authentication protocols enable communicating parties to satisfy themselves mutually about each other's identity and to exchange **session keys**.

For example:



The two important issues in authenticated key exchange are:

1. Confidentiality

Essential identification and session-key information must be communicated in encrypted form to prevent:

- masquerade
- compromise of session keys

This requires the prior existence of secret or public keys.

2. Timeliness

Timeliness is important because of the threat of **message replay**.

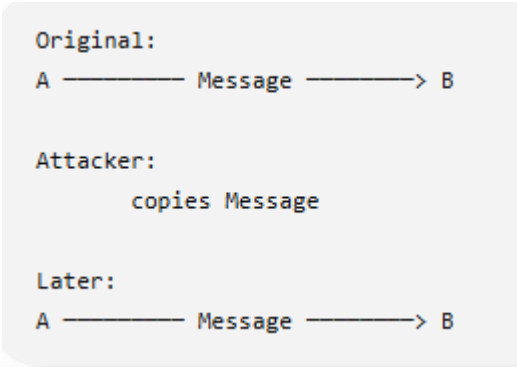
A replay may:

- allow an opponent to compromise a session key,
- allow successful impersonation of another party, or
- disrupt operations by presenting messages that appear genuine but are not.

Replay Attacks

The following are examples of replay attacks.

1. Simple Replay-The opponent simply copies a message and replays it later**.**



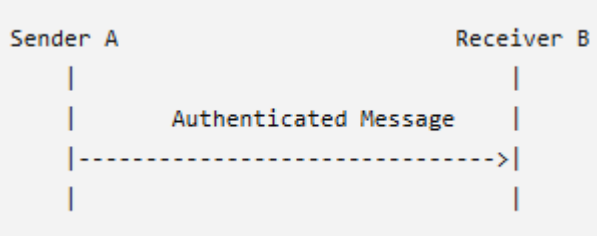
2. Repetition That Can Be Logged- An opponent replays a timestamped message within the valid time window. If both the original and replay arrive within the time window, the incident can be logged.

3. Repetition That Cannot Be Detected-The original message may have been suppressed and therefore never reaches the destination. Only the replayed message arrives, making the replay difficult to detect.

4. Backward Replay Without Modification- The attacker replays a message back to the original sender. This can occur when symmetric encryption is used and the sender cannot easily distinguish between messages sent and messages received based on their content.

One-Way Authentication: In one-way authentication**, only one party needs to be authenticated.**

For example:



The receiver needs assurance about the identity of the sender and the authenticity of the message. The authentication mechanism must also consider the problem of **replay attacks** and the timeliness of the message.

DIGITAL SIGNATURE STANDARD (DSS)

The **Digital Signature Standard (DSS)** approach makes use of a **hash function**. The hash code is provided as input to a signature function along with a **random number k** generated for that particular signature.

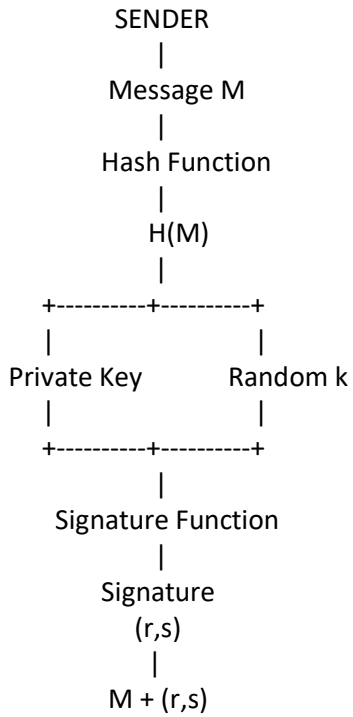
The signature function also depends on:

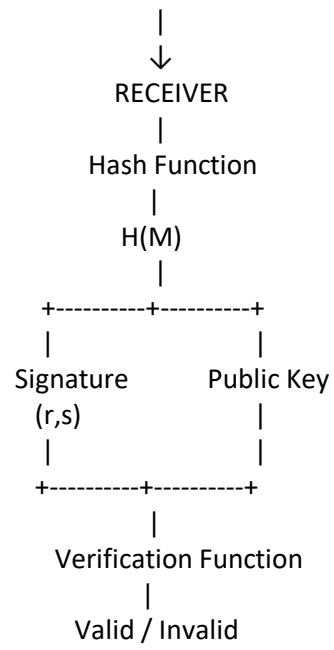
- the sender's private key, and
- a set of parameters known to a group of communicating principals.

The result is a signature consisting of two components: **r and s**.

DSS Approach

The overall process can be represented as:





The receiver generates the hash code of the incoming message. The hash code together with the signature is given to the verification function.The verification function also depends on the **global public key** and the **sender's public key**. If the output of the verification function equals the signature component **r**, the signature is valid.

Digital Signature Algorithm (DSA)-The **DSA** is based on the difficulty of computing **discrete logarithms**. It is based on schemes originally presented by:

- **ElGamal**
- **Schnorr**

The DSA uses:

- global public-key components,
- user's private key,
- user's public key,
- a per-message secret number,
- signing procedure,
- verification procedure.

DSA Parameters

Global Public-Key Components

p-p is a prime number where: $2^{(L-1)} < p < 2^L$

where: **512 ≤ L ≤ 1024**

and **L is a multiple of 64**.

Thus, the bit length is between **512 and 1024 bits**, in increments of 64 bits.

q-q is a prime divisor of (p − 1).

Its bit length is **160 bits**.

g-The parameter g is calculated as: g = h^((p−1)/q) mod p

where h is an integer satisfying the conditions specified in the algorithm.

User's Private Key-The user's private key is represented by: **x**

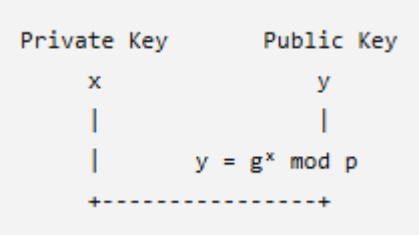
where: **0 < x < q**

x is a random or pseudorandom integer.

Private Key = x

User's Public Key-The user's public key is: **y = g^x mod p**

Therefore:



Per-Message Secret Number

For every signature, DSA uses a random or pseudorandom integer: **k**

such that: **0 < k < q** .This is the **user's per-message secret number**. The hash of the message is also used in generating the signature.

DSA Signing

Let:

- M = message
- H(M) = hash of M
- k = random/pseudorandom per-message secret number
- x = user's private key

The signature consists of two components:

Signature = (r, s)

Step 1: Calculate r

$$r = (g^k \mod p) \mod q$$

Step 2: Calculate s

$$s = [k^{-1}(H(M) + xr)] \mod q$$

Step 3: Signature

$$\boxed{Signature = (r, s)}$$

DSA Verification

At the receiving end:

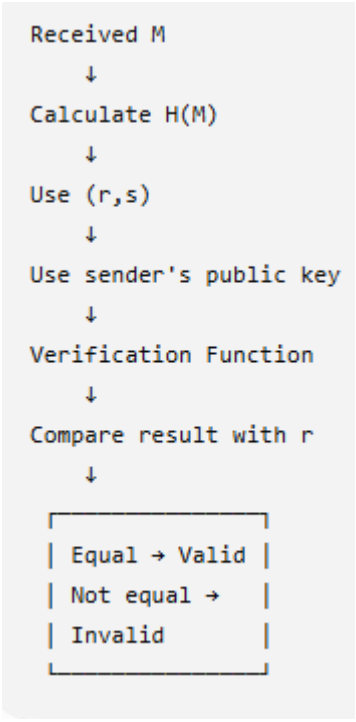
1. The receiver obtains the message.
2. The receiver calculates the hash of the message.
3. The received signature (r,s) is used.
4. The sender's public key is used.
5. The verification function calculates a value.
6. The signature is valid if the resulting value equals **r**.

The important verification equations begin with:

$$w = s^{-1} \mod q$$

and use the message hash, the received signature, the sender's public key, and the global public parameters.

The final verification result is compared with **r**.



1. Global Parameters Selection

- Let prime q = 11
- Let prime p = 23 (notice that p - 1 = 22, which is divisible by q = 11)
- Choose h = 5
- Compute generator $g = 5^{(23-1)/11} \pmod{23} = 5^2 \pmod{23} = 25 \pmod{23} = 2$

Global Parameters: (p, q, g) = (23, 11, 2)

2. Key Generation

- Pick a secret private key x = 5 (where 0 < 5 < 11)
- Compute public key $y = g^x \pmod{p} = 2^5 \pmod{23} = 32 \pmod{23} = 9$
- **Private Key:** x = 5

  Public Key: y = 9

3. Signing a Message (M)

- Suppose the hashed message value is H(M) = 3
- Choose a random number k = 3 (where 0 < 3 < 11)
- Compute first signature component $r = \left(g^k \pmod p \right) \pmod q = \left(2^3 \pmod{23} \right) \pmod{11} = 8 \pmod{11} = 8$
- Compute second signature component $s = \left(k^{-1} \cdot (H(M) + x \cdot r) \right) \pmod q$
  - Find modular inverse of $k = 3 \pmod{11}$, which is 4 (since $3 \times 4 = 12 \equiv 1 \pmod{11}$)
  - $s = 4 \cdot (3 + 5 \cdot 8) \pmod{11} = 4 \cdot (3 + 40) \pmod{11} = 4 \cdot 43 \pmod{11} = 172 \pmod{11} = 7$

Digital Signature Bundle: (r, s) = (8, 7)

4. Verification

The receiver uses p, q, g, public key y=9, message hash H(M)=3, and signature (8, 7) to recompute v and check if v = r.